



Worksheet: 3D Reconstruction with VGGT

In this worksheet, you will explore 3D reconstruction using VGGT. VGGT predicts geometric information directly from images and can be used to recover the three-dimensional structure of a scene.

You can run the model on example images and observe how 3D geometry is inferred from visual input.

Download Example Data

You can download small example datasets that will be used for 3D reconstruction experiments. These datasets provide image sequences and camera information for testing the pipeline.

```
In [1]: # Download a small NeRF example dataset
!wget -q http://cseweb.ucsd.edu/~viscomp/projects/LF/papers/ECCV20/nerf/tiny_r

# Create a data directory
!mkdir -p data
%cd data

# Download and extract example image data
!wget -q http://cseweb.ucsd.edu/~viscomp/projects/LF/papers/ECCV20/nerf/nerf_e
!unzip -q nerf_example_data.zip

%cd ..
```

```
/content/data
/content
```

Prepare Images for VGGT

In this step, you prepare a small set of images that will be used as input to VGGT.

The code selects a subset of RGB images from the NeRF synthetic dataset, resizes them to a fixed resolution, and organises them into a scene directory expected by the model.

```
In [2]: import os
import shutil
import random
from PIL import Image
import matplotlib.pyplot as plt
```

```

SRC = "/content/data/nerf_synthetic/lego/test"
SCENE_DIR = "/content/lego_vgg"
IMAGE_DIR = os.path.join(SCENE_DIR, "images")

os.makedirs(IMAGE_DIR, exist_ok=True)

def is_rgb_image(fname: str) -> bool:
    """
    Return True if filename strictly matches pattern: r_<number>.png
    e.g., r_0.png, r_12.png, r_043.png
    """
    if not fname.startswith("r_") or not fname.endswith(".png"):
        return False
    core = fname[len("r_"):-len(".png")] # part between r_ and .png
    return core.isdigit() and len(core) > 0

# 1. Collect ONLY files of the form r_<number>.png
all_images = [f for f in os.listdir(SRC) if is_rgb_image(f)]
print("Strictly matched r_<number>.png images:", len(all_images))
print(all_images[:20]) # preview first few

if len(all_images) < 8:
    raise ValueError(f"Not enough images matching r_<number>.png, found {len(a)}")

# 2. Randomly select 8
selected = random.sample(all_images, 8)
print("Selected images:", selected)

# 3. Downsample and save
TARGET_SIZE = (320, 320) # change if you want

for f in selected:
    src_path = os.path.join(SRC, f)
    dst_path = os.path.join(IMAGE_DIR, f)
    img = Image.open(src_path).resize(TARGET_SIZE, Image.LANCZOS)
    img.save(dst_path)

print("VGGT scene ready:", IMAGE_DIR)
print("Total images saved:", len(os.listdir(IMAGE_DIR)))

# 4. Visualize the selected images
plt.figure(figsize=(16, 8))

for i, f in enumerate(selected):
    img_path = os.path.join(IMAGE_DIR, f)
    img = Image.open(img_path)
    plt.subplot(2, 4, i + 1)
    plt.imshow(img)
    plt.title(f)
    plt.axis("off")

plt.tight_layout()

```

```
plt.show()
```

Strictly matched r_<number>.png images: 200

['r_89.png', 'r_57.png', 'r_59.png', 'r_96.png', 'r_141.png', 'r_165.png', 'r_55.png', 'r_154.png', 'r_101.png', 'r_158.png', 'r_134.png', 'r_151.png', 'r_12.png', 'r_99.png', 'r_5.png', 'r_159.png', 'r_72.png', 'r_86.png', 'r_118.png', 'r_188.png']

Selected images: ['r_98.png', 'r_188.png', 'r_195.png', 'r_55.png', 'r_68.png', 'r_154.png', 'r_99.png', 'r_151.png']

VGGT scene ready: /content/lego_vggt/images

Total images saved: 8

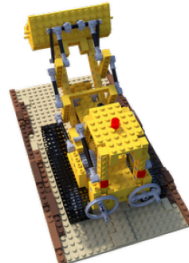
r_98.png



r_188.png



r_195.png



r_55.png



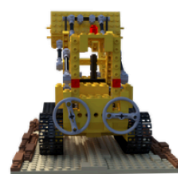
r_68.png



r_154.png



r_99.png



r_151.png



Install VGGT

You can install VGGT and its required dependencies.

```
In [3]: !git clone https://github.com/facebookresearch/vggt.git
        %cd /content/vggt

        !pip install -q -r requirements_demo.txt --progress-bar off
```

```
Cloning into 'vggt'...
remote: Enumerating objects: 1265, done.
remote: Total 1265 (delta 0), reused 0 (delta 0), pack-reused 1265 (from 1)
Receiving objects: 100% (1265/1265), 64.94 MiB | 22.14 MiB/s, done.
Resolving deltas: 100% (579/579), done.
/content/vggt
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
  Building wheel for lightglue (pyproject.toml) ... done
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.
mcp 1.23.3 requires pydantic<3.0.0,>=2.11.0, but you have pydantic 2.10.6 which is incompatible.
google-adk 1.20.0 requires websockets<16.0.0,>=15.0.1, but you have websockets 14.2 which is incompatible.
```

Run VGGT Reconstruction

You can now run VGGT on the prepared scene to perform 3D reconstruction.

```
In [4]: !python demo_colmap.py \
  --scene_dir="/content/lego_vggt" \
  --fine_tracking \
  --max_query_pts 512

/usr/local/lib/python3.12/dist-packages/lightglue/lightglue.py:24: FutureWarning: `torch.cuda.amp.custom_fwd(args...)` is deprecated. Please use `torch.amp.custom_fwd(args..., device_type='cuda')` instead.
  @torch.cuda.amp.custom_fwd(cast_inputs=torch.float32)
Arguments: {'scene_dir': '/content/lego_vggt', 'seed': 42, 'use_ba': False, 'max_reproj_error': 8.0, 'shared_camera': False, 'camera_type': 'SIMPLE_PINHOLE', 'vis_thresh': 0.2, 'query_frame_num': 8, 'max_query_pts': 512, 'fine_tracking': True, 'conf_thres_value': 5.0}
Setting seed as: 42
Using device: cuda
Using dtype: torch.float16
Downloading: "https://huggingface.co/facebook/VGGT-1B/resolve/main/model.pt" to /root/.cache/torch/hub/checkpoints/model.pt
100% 4.68G/4.68G [00:48<00:00, 104MB/s]
Model loaded
Loaded 8 images from /content/lego_vggt/images
/content/vggt/demo_colmap.py:75: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(dtype=dtype):
Converting to COLMAP format
Saving reconstruction to /content/lego_vggt/sparse
```

Visualise the Reconstruction

You can now visualise the reconstruction results produced by VGGT.

```
In [5]: !pip install -q open3d --progress-bar off
```

```
In [6]: import struct
import numpy as np
import open3d as o3d

def read_points3D_binary(path):
    with open(path, "rb") as f:
        num = struct.unpack("Q", f.read(8))[0]
        pts, cols = [], []
        for _ in range(num):
            f.read(8)
            xyz = struct.unpack("ddd", f.read(24))
            rgb = struct.unpack("BBB", f.read(3))
            f.read(8)
            tlen = struct.unpack("Q", f.read(8))[0]
            f.read(tlen * 8)
            pts.append(xyz)
            cols.append(rgb)
        return np.array(pts), np.array(cols)/255.

points, colors = read_points3D_binary("/content/lego_vggg/sparse/points3D.bin")

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(points)
pcd.colors = o3d.utility.Vector3dVector(colors)

o3d.visualization.draw_plotly([pcd])
```

Output hidden; open in <https://colab.research.google.com> to view.

Reconstruct a Scene from Your Own Images

You can upload your own set of images and run the 3D reconstruction pipeline on them.

```
In [7]: import os
from google.colab import files
from PIL import Image

# Target directory for VGGT
SCENE_DIR = "/content/my_scene"
IMAGE_DIR = os.path.join(SCENE_DIR, "images")

os.makedirs(IMAGE_DIR, exist_ok=True)
```

```

# 1 Upload images
uploaded = files.upload()

# 2 Save & downsample
TARGET_SIZE = (320, 320)

for name, data in uploaded.items():
    dst = os.path.join(IMAGE_DIR, name)

    # Save file
    with open(dst, "wb") as f:
        f.write(data)

    # Downsample
    img = Image.open(dst)
    img = img.resize(TARGET_SIZE, Image.LANCZOS)
    img.save(dst)

print("Uploaded and downsampled images:", os.listdir(IMAGE_DIR))
print("Total images:", len(os.listdir(IMAGE_DIR)))
print("Final resolution:", TARGET_SIZE)

```

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```

Saving 000.png to 000.png
Saving 006.png to 006.png
Saving 010.png to 010.png
Saving 011.png to 011.png
Saving 014.png to 014.png
Saving 015.png to 015.png
Saving 018.png to 018.png
Saving 019.png to 019.png
Uploaded and downsampled images: ['019.png', '018.png', '015.png', '014.png',
'006.png', '011.png', '000.png', '010.png']
Total images: 8
Final resolution: (320, 320)

```

In [8]: `!python demo_colmap.py \`
`--scene_dir="/content/my_scene" \`
`--max_query_pts 512`

```

/usr/local/lib/python3.12/dist-packages/lightglue/lightglue.py:24: FutureWarning: `torch.cuda.amp.custom_fwd(args...)` is deprecated. Please use `torch.amp.custom_fwd(args..., device_type='cuda')` instead.
  @torch.cuda.amp.custom_fwd(cast_inputs=torch.float32)
Arguments: {'scene_dir': '/content/my_scene', 'seed': 42, 'use_ba': False, 'max_reproj_error': 8.0, 'shared_camera': False, 'camera_type': 'SIMPLE_PINHOLE', 'vis_thresh': 0.2, 'query_frame_num': 8, 'max_query_pts': 512, 'fine_tracking': True, 'conf_thres_value': 5.0}
Setting seed as: 42
Using device: cuda
Using dtype: torch.float16
Model loaded
Loaded 8 images from /content/my_scene/images
/content/vggg/demo_colmap.py:75: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(dtype=dtype):
Converting to COLMAP format
Saving reconstruction to /content/my_scene/sparse

```

```

In [9]: import struct
import numpy as np
import open3d as o3d

def read_points3D_binary(path):
    with open(path, "rb") as f:
        num = struct.unpack("Q", f.read(8))[0]
        pts, cols = [], []
        for _ in range(num):
            f.read(8)
            xyz = struct.unpack("ddd", f.read(24))
            rgb = struct.unpack("BBB", f.read(3))
            f.read(8)
            tlen = struct.unpack("Q", f.read(8))[0]
            f.read(tlen * 8)
            pts.append(xyz)
            cols.append(rgb)
        return np.array(pts), np.array(cols)/255.

points, colors = read_points3D_binary("/content/my_scene/sparse/points3D.bin")

pcd = o3d.geometry.PointCloud()
pcd.points = o3d.utility.Vector3dVector(points)
pcd.colors = o3d.utility.Vector3dVector(colors)

o3d.visualization.draw_plotly([pcd])

```

Output hidden; open in <https://colab.research.google.com> to view.

In [9]: